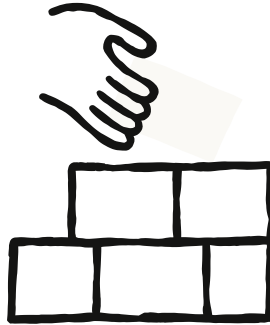


Customize Claude Code with mods

Anthropic · 2026-10-01 · 原文 / Source



Change how Claude Code behaves and looks with a few lines of TypeScript.

Today we're introducing mods, small TypeScript functions that change how Claude Code works. A mod can rewrite a prompt, add new UI, replace a built-in feature, or add entirely new functionality. You can write a mod yourself, or ask Claude Code to write one for you. Mods ship inside plugins, so you install and share them like any plugin. They work in the Claude Code CLI and desktop app.

Mods run with the same access to your machine as Claude Code itself. They aren't sandboxed, and you should only install mods from sources you trust, the same way you'd install any code on your computer.

To see what mods can do, read our guide to building your first mod.

Why we built mods

Developers have asked for more control over how Claude Code works, without waiting for us to ship a feature. Hooks helped give users some of this control, but hooks can't rewrite events, draw new UI, or replace features. Mods can.

We want Claude Code to feel like yours, so you can shape it to the way you work. We shared the design for mods on GitHub before launch to get feedback from developers. Thank you to everyone who weighed in.

How mods work

Each time Claude Code does something, it emits an event. Examples include calling a tool, asking for permission, and drawing part of the screen. A mod is a function that hooks into one of these events. A mod can run before the event, after it, or instead of it. It can also wrap the event, running code both before and after. With one function, a mod can:

- Rewrite a prompt before it reaches the model.
- Block, rewrite, or retry a tool call.
- Approve or deny a permission request.
- Redact secrets from tool output before Claude reads it.

A mod can also change what you see. It can edit or replace part of the interface Claude Code draws, like a tool result or a question from Claude. It can add buttons and inputs, and other mods can respond when you press them. Today, a mod can target the terminal, the desktop app, or both.

When several mods hook the same event, they run in the order they load. The first mod to load sees the event first and the result last. This lets you stack mods from different authors.

You can also use Claude Code to mod Claude Code. Ask Claude to create a mod, and it can write the TypeScript, install it, and hot reload it in your session.

Swap built-in features for your own

Some built-in features of Claude Code now ship as mods. For example, the built-in `/diff` feature is now a mod, so you can turn it off (in `/plugin`) or replace it with your own version. We plan to move more built-in features to mods over time, so you can pare Claude Code down to a small core and add back only what you want.

Mods for teams and enterprises

Mods ship inside plugins, so your existing plugin controls apply. Admins can allow or block plugin marketplaces. On Team and Enterprise plans, an owner sets this in the admin console. On Claude API and third-party API plans, admins push managed settings to users' machines.

On Team and Enterprise plans, and on any machine with managed settings, a built-in mod called `sec-default` ("security default") loads first. It stops mods that users install from doing risky things, like overriding your permission deny rules. You can view the source code to see what it restricts. Admins can load their own mods first instead. If you do, add `sec-default` to your list to keep its restrictions.

Teams can also use mods to build their own controls and functionality. For example:

- **CI/CD status:** a mod can show your pipeline's status in a pane beside the conversation, and update it as builds pass or fail.
- **Production safeguards:** a mod can require confirmation before any command touches production config.
- **Audit logging:** a mod that loads first can record every call that every other mod makes.

Getting started

Mods are available today in the Claude Code CLI and desktop app. Install plugins that include mods from the Claude directory or by running `/plugin` in the CLI. To share a mod, package it in a plugin and submit it to the directory.

To build your own mods, read our getting started guide or see the documentation.